

BD3-LM Reference Code Summary & Implementation Comparison

2026-07-16 · Reference: `github.com/kuleshov-group/bd3lms` (Arriola et al., "Block Diffusion: Interpolating Between Autoregressive and Diffusion Language Models", ICLR 2025) · Local: `models_seq/bd_models.py`, `models_seq/bd_trainer.py`, `main_bd.py`

What "passed the comparison verification" means, precisely. The reference repository was cloned and the five core components of the mask kernel (attention mask, vectorized training, SUBS loss, first-hitting sampler, semi-AR generation loop) were compared side-by-side at the code level; **all five compute the same mathematics and algorithms as the original**. What this verification does not mean: (1) no numerical-equivalence test was run (loading identical weights into both implementations and diffing outputs is impossible — the two codebases use different vocabularies and conditioning); (2) it does not mean every feature of the original was ported — EMA, clipped schedules, etc. were deliberately dropped and are all listed in §4; (3) the graph (uniform-state) kernel was never meant to be a bd3lms port — it is this repository's own design applying the `seq_models.py` loss per block (per CLAUDE.md instructions).

1. The problem BD3-LM solves, and the core idea

Background. Autoregressive (AR) models achieve high likelihood and fast KV-cached decoding but must emit tokens strictly one at a time. Discrete diffusion (e.g. MDLM) allows parallel generation but lags in likelihood and is tied to a fixed-length canvas, making variable-length generation awkward. BD3-LM **interpolates between the two at the block level**.

Block decomposition. The sequence is split into blocks of size B ; the likelihood factorizes autoregressively across blocks while each block's interior is modeled by masked discrete diffusion:

$\log p(x) = \sum_b \log p(x^b \mid x^{\{<b\}})$ ← across blocks: autoregressive
 $p(x^b \mid x^{\{<b\}})$: within a block: discrete diffusion (NELBO conditioned on previous blocks)

- $B = 1$ recovers pure AR; $B = \text{sequence length}$ recovers pure diffusion — **block size is the AR↔diffusion interpolation knob**.
- Generation is semi-autoregressive: finish one block by diffusion, commit it, append the next noised block. This structure is what enables **variable-length generation** and a **cross-block KV cache**.

2. Key parts of the reference code (by file / function)

2.1 Vectorized training — all blocks in one forward pass

`diffusion.py :: _forward_pass_diffusion` . A naive implementation would need one forward per block (cost $\times$ number of blocks). BD3-LM instead feeds the **concatenation $[x_t \parallel x_0]$ (length $2n$)** in a single pass and controls information flow with a specialized attention mask. The noise level t is **sampled independently per block** (`_sample_t` : one uniform sample per block, then `repeat_interleave`).

2.2 The specialized attention mask — OR of three sub-masks

`models/hf/modeling_bd3lm.py :: block_diff_mask` . Each query/key is classified by which half it lives in (`x0_flag`) and its block index:

`M_BD` (block diagonal) : same half & same block → self-attention inside a noised block
`M_OBC` (offset block causal) : x_t query → x_0 keys of strictly earlier blocks → clean context conditioning
`M_BC` (block causal) : x_0 query → x_0 keys of earlier-or-equal blocks → updates clean (cacheable) states
`mask = M_BD | M_OBC | M_BC`

2.3 Loss — MDLM-style NELBO with SUBS parameterization

`diffusion.py :: _subs_parameterization, _loss` . The denoiser predicts $\hat{x}_0$ (logits over the vocabulary). SUBS (a) **forces the MASK token's probability to zero** (`logits[:, :, mask_index] = -inf`), and (b) **carries over already-revealed (unmasked) positions** as one-hot (their log-prob becomes 0 → no loss contribution). Under the log-linear schedule the NELBO reduces to a simple weighted CE:

$$L = E_{t \sim U[\epsilon, 1]} [-(1/t) \cdot \sum_{\{ \text{masked } i \}} \log p_{\theta}(x_{0^i} \mid x_t)] \quad (t \text{ independent per block})$$

2.4 Sampling — semi-AR loop + first-hitting sampler

`diffusion.py :: _semi_ar_sampler, _ddpm_caching_update` . The outer loop appends one block at a time; the inner loop denoises the current block. The first-hitting sampler (the equivalent, faster sampler of Zheng et al. 2025) jumps directly to the time at which the next token is revealed:

```
u ~ U(0,1); t ← t · u^(1/num_masked)      ← first-hitting time update
p_x0 = softmax(denoiser(x)); pick ONE masked position uniformly, sample it from p_x0
```

→ one forward pass per revealed token; a block finishes in exactly B reveals. On top of this sit nucleus (top-p) sampling, KV-cache storage once a block completes, and EOS/entropy-based stopping (`_check_stop_conds`).

2.5 Variance reduction — the paper's key training techniques

- **Clipped noise schedules** (`config.algo.var_min=True` , on by default): t is sampled only from an interval $[\epsilon_{\min}, \epsilon_{\max}] \subset [0,1]$. During training the interval that **minimizes the variance** of the validation loss is searched periodically (`_clipped_schedule_search` , delta 0.05). This is a headline contribution of the paper — it substantially reduces gradient variance and improves perplexity, and **matters most at small block sizes**.
- **Antithetic t sampling**: stratifies t across the batch and blocks to reduce estimator variance.
- **EMA** (0.9999): an exponential moving average of the weights is used for evaluation and generation.

2.6 Backbone — DDiT (rotary + adaLN)

`modeling_bd3lm.py :: DDiTBlock` . Pre-LN transformer with **rotary position embeddings**; the noise level σ is embedded by a `TimestepEmbedder` and injected via **adaLN** (zero-init; 6 modulations per block: shift/scale/gate $\times$ attn/MLP). For masked diffusion, however, masked tokens themselves reveal the noise level, so **time conditioning is effectively unnecessary** (stated in the paper), and the conditioning vector c is broadcast once per sequence.

3. Component-by-component comparison (mask kernel = the BD3-LM port)

Component	Reference (bd3lms)	Local (bd_models.py)	Verdict
Training attention mask	<code>block_diff_mask</code> : M_BD M_OBC M_BC (flex-attention mask_mod)	<code>get_block_diff_mask</code> (L61): OR of the same three sub-masks, materialized as a cached boolean tensor	Match — the three sub-mask predicates are literally identical
Vectorized training	$[x_t \parallel x_0]$ concat, per-block $t \sim U$ then <code>repeat_interleave</code>	<code>_forward_mask</code> (L416): same concat, <code>_sample_t_blocks</code> → <code>repeat_interleave</code>	Match
Loss (NELBO)	<code>loss_scale</code> = $-1/t$ (log-linear); SUBS carry-over restricts contribution to masked positions; normalized by valid-token count	$-(1/t) \cdot \log p_\theta(x_0)$ applied explicitly to masked positions only; normalized by the loss-mask sum	Match — carry-over (reference) vs. explicit masking (local) are two implementations of the same value
SUBS constraint	<code>logits[MASK]</code> = $-\infty$; unmasked pinned to one-hot	<code>logits[... , MASK]</code> = $-1e9$; unmasked positions excluded from both loss and sampling updates	Match (behaviorally)
First-hitting sampler	$t \leftarrow t \cdot u^{(1/\text{num_masked})}$ (u a batch-shared scalar); samples the whole block, keeps one uniformly-chosen masked position	<code>_denoise_block_mask</code> (L689): same time update (u per sample); picks one masked position uniformly, then samples it from p_{x_0}	Match — the committed token's distribution is identical; per-sample u only adds within-batch independence
Semi-AR generation	<code>_semi_ar_sampler</code> : append block → denoise → (EOS/entropy stop)	<code>plan</code> (L570): append block → denoise → scan for dst (hit) / END (miss)	Structure matches (stop rule replaced for the task, §4-A)
Time conditioning	Effectively unnecessary for masked diffusion (masks self-identify)	Off by default for the mask kernel (opt-in via <code>bd_time_cond</code>); forced on for the graph kernel	Match (follows the paper's own argument)

4. Where the local implementation differs — complete list

A. Domain adaptation (language modeling → OD route planning)

Item	Details
Vocabulary / states	BPE subwords → road-graph vertex ids ($V=1,390$) plus <code>END</code> ($=V$), <code>PAD</code> ($=V+1$), <code>MASK</code> ($=V+2$). Vertex embeddings initialized from <code>node2vec</code> .
Canvas / length spec	<code>[dst, ori, v1, ..., v_L=dst, END...END PAD...PAD]</code> — the remainder of the block containing the path's final token is filled with <code>END</code> and is a training target ; all later blocks are <code>PAD</code> and excluded from the loss (<code>build_canvas</code> ; boundary cases asserted by the smoke test). A project-specific design absent from the reference.
OD conditioning	Two channels: (1) <code>dst-ori</code> pinned as clean in-context tokens at canvas positions 0–1; (2) <code>od_mlp(emb(ori); emb(dst))</code> added to the adaLN conditioning vector. Condition dropout 0.1 + a learned <code>null_cond</code> → unconditional generation and CFG supported (default $w=1$, CFG off). The reference only generates unconditionally.
Stop rule	Instead of the reference's EOS/entropy/likelihood stopping (<code>_check_stop_conds</code>), the committed block is scanned: dst appears → hit (success stop) ; END/PAD/MASK appears → miss . No length model — length is emergent.
Post-processing (eval)	Missed paths get <code>refine_nx</code> (shortest-path patch) / <code>refine_es</code> (<code>eval_shortest.refine()</code>) variants scored alongside. The current experiment reports raw (no post-processing) only.

B. Structural differences (similar behavior, different implementation)

Item	Reference	Local
Position encoding	Rotary (relative)	Learned absolute position embedding (<code>pos = arange(S) % n</code> — the x_t/x_0 halves share positions)

Conditioning granularity	One c per sequence (σ embedding), broadcast	c is per token (per-block t + OD embedding) — needed to inject per-block noise levels and the OD channel simultaneously
Attention kernel	FlexAttention + torch.compile (torch 2.x)	Manual matmul + masked_fill (torch 1.12 compatible; mathematically identical)
Output head	No weight tying; DDitFinalLayer (adaLN)	Same structure (final adaLN + zero-init head)

C. Features present in the reference but removed locally

Feature	Role in the reference	Impact assessment
Clipped noise schedule (var_min search)	Adaptively searches the t-interval $[\epsilon_{\min}, \epsilon_{\max}]$ minimizing validation-loss variance. A headline contribution; matters most at small block sizes	Local uses fixed $t \in [1e-3, 1]$. The one difference to keep in mind when reading the block-size sweep — small blocks (1, 2) may underperform relative to the original recipe. Within-sweep comparisons remain fair (identical conditions across all sizes)
EMA (0.9999)	Weight moving average used at eval/generation	Not implemented. The existing blk4 baseline (hit 0.952) was also trained without EMA — the sweep stays internally consistent
Antithetic t sampling	Stratified t to reduce gradient variance	Deliberately removed (stated in the module docstring). Slightly noisier loss estimates
KV cache / nucleus sampling	Generation speed / quality-diversity trade-off	Speed/temperature features — no effect on accuracy evaluation (full-softmax sampling = temp 1.0)
Fixed-step mask sampler	Alternative to first-hitting (T-step discretization)	Only first-hitting kept (it is the equivalent, faster one)

D. Local-only addition — the graph (uniform-state) kernel

This kernel does not exist in bd3lms; it is a **composition, not a port**: BD3-LM's block machinery (attention masks, $[x_t \parallel x_0]$ training, semi-AR generation) is kept, while the within-block noising and loss are replaced by this repository's uniform-state CTMC diffusion from `seq_models.py` (Destroyer/Restorer), per block-diffusion-loss.pdf and CLAUDE.md.

- Noising: $Q_t = \exp((A' - D')\beta_t)$ — a CTMC built from the road-graph adjacency; the state space is $V+1$ with an added `<end>` state (virtual `<end>` node of degree `bd_eos_deg`).
- Loss: D3PM ELBO = $KL(q(x_{t-1}|x_t, x_0) \parallel p_\theta) + CE + \text{connectivity regularizer}$ (the same math as `Restorer.forward`, applied per block under the loss mask).
- Generation: full `max_T` (100)-step reverse per block with an MC posterior approximation, followed by an adjacency-constrained decode.
- Time conditioning forced on (uniform noise is not self-identifying from x_t alone).

5. Summary

- The mask kernel follows BD3-LM's mathematics (mask · loss · sampler) exactly** — all five components in §3 verified by side-by-side code comparison.
- The differences fall into three buckets: **(A) task adaptation** (OD conditioning · length spec · stop rule — required by the project spec), **(B·C) removed training conveniences** (EMA · clipped schedules · antithetic — second-order effects on accuracy, none on within-sweep comparisons), and **(D) the added graph kernel** (an original design outside bd3lms).
- One caveat for interpreting results: the **absence of clipped schedules** may make small block sizes look worse than the original recipe would. Trends across block sizes remain valid, but avoid absolute claims like "blk1 equals the paper's AR baseline."